Cahier des charges

1. Définition of Done

Introduction

Mindiff est une application mobile d’accompagnement sportif, accompagnée d’une intelligence artifcielle capable d'analyser les mouvements de l'utilisateur via la caméra. L'application proposera des exercices adaptés selon le profil de l'utilisateur et suivra ses progrès pour améliorer sa performance.

1- Phase de développement terminée

Le code est écrit en respectant les standards et bonnes pratiques de développement.

Les fonctionnalités sont implémentées conformément aux spécifications (User Stories, Figma, maquettes) pré-définies par les développeurs mobile auparavant.
Aucune dette technique ou workaround temporaire non documenté.
Le code est **documenté** (commentaires clairs, documentation des méthodes et classes si nécessaire).
Les dépendances sont bien gérées (pas de paquets inutiles, versions à jour pour tous les membres de l’équipe).

2. Tests & Qualité

Tests unitaires écrits et validés avec un taux de couverture minimal défini.
Tests d'intégration effectués pour vérifier l'interaction entre les modules.
Tests UI/UX manuels effectués pour s’assurer de la cohérence graphique et de l’ergonomie.
Tests de performance effectués pour éviter les ralentissements et crashs.
Tests de régression passés pour s'assurer qu'aucune fonctionnalité existante n'est cassée.
Tests sur différents devices et OS (Android/iOS, différentes résolutions et versions).

3. Sécurité & Performance

Gestion correcte des **données utilisateur** (chiffrement, stockage sécurisé, RGPD).
Authentification et gestion des sessions sécurisées (ex. JWT, OAuth).
Pas de **failles de sécurité connues** (vulnérabilités OWASP vérifiées).
Gestion correcte des **permissions** (caméra, GPS, stockage, etc.).
Temps de chargement optimisé (<3 sec au démarrage, <1 sec pour les actions critiques).

4. CI/CD & Automatisation

Code validé via un pipeline CI/CD (linting, build, tests).
Aucun **build cassé** dans la pipeline.
Déploiement possible en **un clic** ou via **GitLab CI/CD**.
App disponible en version **beta/test interne** via TestFlight / Firebase App Distribution.

5. UX/UI & Accessibilité

Interface **responsive** et adaptée aux guidelines Apple/Google.
Expérience utilisateur fluide (animations, feedback visuel).

Accessibilité testée (access handicap).
Aucune **incohérence graphique** par rapport aux maquettes.

6. Documentation & Communication

Documentation à jour sur **les APIs, architecture et modules**.
README.md détaillé pour expliquer l'installation et l'utilisation.
Changements et mises à jour notifiés à l'équipe (Slack, Notion, GitLab, etc.).

7. Validation & Déploiement

L'application est prête à être publiée sur Google Play Store / App Store.

Vérification du respect des guidelines Apple et Google avant soumission.
Tests finaux réalisés sur des appareils physiques avant la mise en ligne

2. Justification des technologies

Objectif de la Mindiff

Créer une application mobile complète qui permet aux utilisateurs de suivre un programme sportif personnalisé, d'améliorer leurs performances avec une balance connectée, une caméra d'analyse de mouvements, et de bénéficier de conseils nutritionnels adaptés.

La solution doit être : multiplateforme, scalable, fiable, connectée à des objets intelligents, et sécurisée.

Technologies choisies et justification

Framework Mobile : Flutter

Cross-platform natif, permet de cibler iOS & Android avec une seule base de code. Performance élevée, communauté active, intégration facile avec Firebase.

Backend & Authentification : Firebase

Firebase Auth pour login email/SSO + 2FA. Functions serverless pour logique backend sécurisée, rapide à déployer.

Base de données : Firestore

NoSQL scalable, mise à jour en temps réel, facile à connecter à Flutter et Firebase Auth.

Analyse de mouvements : IA intégrée

Modèle embarqué pour retour en temps réel, même hors ligne. Respect de la vie privée car les vidéos ne quittent pas l'app.

Cloud Storage : Firebase Storage

Stockage sécurisé des vidéos et données utilisateurs. Accès restreint via Firebase Auth. CDN Google pour performance.

Cohérence de l'ensemble

L'ensemble des technologies est cohérent, bien intégré, et permet un développement rapide avec un bon niveau de sécurité et de scalabilité. Firebase + Flutter forment une stack moderne parfaitement adaptée aux besoins de Mindiff.

3. Users needs

Utilisateurs

Utilisateurs	Description	Acronymes
Utilisateur Inscrit	Utilisateur ayant un compte mais ne payant pas, il peut créer un programme personnel	UI
Utilisateur Premium	Utilisateur ayant un abonnement, il a accès à l'ensemble de l'application	UP
Utilisateur non-inscrit	Utilisateur qui n'est pas inscrit, il a accès au programme sportif de base	UNI

Modèles

UF (Unité Fonctionnelle)	UP (Unité de Processus)	US (User Story)	UNI	UI	UP
1 - Connexio	1.1 – Se connecter	En tant qu'utilisate	✅	✅	✅

n	via courriel & mot de passe	ur, je veux pouvoir me connecter avec mon adresse e-mail et un mot de passe afin d'accéder à mon compte.			
	1.2 – Se connecter via Single Sign-On (SSO)	En tant qu'utilisateur, je veux pouvoir me connecter avec mon compte Google/Apple/Facebook pour plus de simplicité.	❌	✅	✅
	1.3 – S'identifier avec vérification double facteur	En tant qu'utilisateur, je veux pouvoir activer la double authentification pour	❌	✅	✅

		sécuriser mon compte.			

	1.4 – Se connecter sur le site web	En tant qu'utilisateur, je veux pouvoir me connecter depuis un navigateur web pour accéder à mon compte.	❌	✅	✅

2 - Inscription	2.1 – S'inscrire via courriel & mot de passe	En tant que nouvel utilisateur, je veux pouvoir créer un compte avec mon adresse e-mail et un mot de passe pour commencer à utiliser l'application.	✅	✅	✅

	2.2 – Possibilité d'activer la double authentification	En tant qu'utilisateur, je veux activer une double authentification pour mieux protéger mon compte.	❌	✅	✅

	2.3 – Choisir son niveau d'abonnement/paiement	En tant qu'utilisateur, je veux pouvoir choisir entre une version gratuite ou premium pour accéder à différentes fonctionnalités.	❌	✅	✅

	2.4 – Lire et accepter les termes et conditions	En tant qu'utilisateur, je veux pouvoir consulter et accepter	✅	✅	✅

		les conditions d'utilisation avant de finaliser mon inscription.			

	2.5 – Renseignement des informations personnelles	En tant qu'utilisateur, je veux fournir mes informations de base (âge, taille, poids, niveau sportif) afin de personnaliser mon expérience.	❌	✅	✅

3 - Programme d'entraînement	3.1 – Accéder à un programme de base	En tant qu'utilisateur non inscrit, je veux pouvoir tester un programme sportif basique	✅	❌	❌

		avant de créer un compte.			

	3.2 – Créer un programme personnalisé	En tant qu'utilisateur inscrit, je veux pouvoir créer un programme sur mesure pour suivre ma progression.	❌	✅	✅

	3.3 – Accéder à des entraînements premium	En tant qu'utilisateur premium, je veux avoir des entraînements exclusifs adaptés à mes objectifs.	❌	❌	✅

4 - Suivi des performances	4.1 – Enregistrer les séances d'entraînement	En tant qu'utilisateur, je veux suivre mes entraînements pour voir mes progrès.	❌	✅	✅
	4.2 – Suivi via balance connectée	En tant qu'utilisateur premium, je veux voir l'évolution de mon poids et de ma composition corporelle grâce à la balance connectée.	❌	❌	✅
	4.3 – Analyse des mouvements via caméra	En tant qu'utilisateur premium, je veux avoir un retour précis sur	❌	❌	✅

		ma posture et mes mouvements pour m'améliorer.			
5 – Menu personnaliser	5.5 – Accéder à un menu de basa	En tant qu'utilisateur premium je peux accéder à des menus pré-créer	❌	❌	✅
	3.2 – Créer un programme personnalisé	En tant qu'utilisateur premium, je veux créer un menu personnalisé à mes objectifs et mes mensurations	❌	❌	✅

4. Intérgation équipe mobile

Bienvenue chez Mindiff !

Tu rejoins la team Application Mobile de Mindiff, notre application sportive complète qui combine entraînement personnalisé, nutrition adaptée, balance connectée et analyse de mouvements via IA.

Présentation rapide des équipes

- Team Balance Connectée : Communication Bluetooth et wifi avec l'app
- Team Caméra / Analyse IA : Développement du modèle IA de détection de mouvements et intégration caméra
- Team Application Mobile : Développement de l'application Flutter, gestion du compte utilisateur, création de programme, interface nutrition, intégration des données balance/caméra, etc.

Stack technique – Application Mobile

Domaine	Outils/Technos utilisés
Framework Mobile	Flutter
Backend	Firebase (Authentication & Functions)
Base de données	Firebase Firestore
Cloud Storage	Firebase Storage
IA mouvement	Modèle IA embarqué côté app
UI/UX	Figma
Suivi Projet	Jira
Documentation	Jira

Ton rôle dans la team

En tant que développeur Flutter, tu seras amené à :

- Implémenter les écrans de l'app (training, nutrition, progrès…)
- Intégrer les flux de données de la balance et de la caméra
- Travailler en binôme avec le designer UI/UX

- Participer aux réunions
- Optimiser les performances de l'app sur iOS & Android

Tes débuts en 5 étapes

Étape	Description
1. Accès outils	Création de ton compte sur : GitHub, Firebase Console, Jira, Figma
2. Setup local	Cloner le repo GitHub + setup Flutter : flutter pub get, flutter run
3. Connexion Firebase	Ajout des fichiers google-services.json et GoogleService-Info.plist
4. Découverte Jira	Lecture de la documentation interne
5. Premier ticket	Ticket simple : correction UI ou intégration de données

Outils de communication

Usage	Outil
Communication	Discord
Réunions d'équipe	Discord ou présentiel
Fichiers partagés	Jira
Design UI	Figma
Gestion de projet	Jira

Notre team

Nom	Rôle
Audran Parthiot	Equipe Caméra
Swan Marin	Equipe Caméra
Balthazar Supiot	Equipe balance connectée
Alexis Angelo	Equipe balance connectée
Garance Rivassou	Equipe developpement
Hugo Thomazeau	Equipe developpement
Nicolas Garde	Equipe developpement
Clara Choupin	Equipe developpement

Bon démarrage et bienvenue dans l'aventure Mindiff !

5. Méthodologie de développement

Introduction

Dans le cadre du développement de l'application mobile **Mindiff**, une solution de suivi fitness connectée à une balance intelligente, nous avons structuré notre travail autour d'une **méthodologie hybride**, combinant des éléments d'**Agile** et de **Kanban**.

Ce choix vise à travailler de manière **efficace, adaptable et collaborative**, tout en gardant une vision claire de l'avancement.
Ce document expose les fondements de cette approche, les raisons de son adoption, et sa mise en œuvre dans le cadre du projet.

---

1. Principes de la méthode Agile

La méthode **Agile** repose sur **quatre valeurs fondamentales** :

- Les individus et leurs interactions plus que les processus et les outils

- Un logiciel opérationnel plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

Ces valeurs sont complétées par **douze principes**, dont les plus pertinents pour notre projet :

- Livraison fréquente de fonctionnalités opérationnelles
- Implication continue des utilisateurs ou parties prenantes
- Adaptabilité face aux changements
- Coopération quotidienne entre les membres de l'équipe
- Amélioration continue des processus

Cette philosophie permet une organisation **souple, réactive**, centrée sur la valeur pour l'utilisateur final, tout en favorisant **la communication et la transparence**.

---

2. Présentation de Kanban

Kanban est un système visuel de gestion de flux de travail, utilisé initialement dans l'industrie et aujourd'hui adapté au développement logiciel.

Principes de base :

- **Visualiser le flux de travail** via un tableau à colonnes (ex. : À faire / En cours / Fait)
- **Limiter le travail en cours (WIP)** pour éviter la surcharge et garantir la fluidité
- **Mesurer et améliorer en continu** : identifier les blocages, ajuster les processus

Cette approche permet un **suivi en temps réel** des tâches, une **détection rapide des blocages**, et une **optimisation de la charge** selon les ressources disponibles.

---

3. Mise en œuvre de la méthode hybride Agile / Kanban

Organisation du travail

- Utilisation d'un **tableau Kanban** partagé (Trello, Jira, etc.)
- Découpage des tâches en unités fonctionnelles : développement, test, design, intégration
- Trois colonnes principales : **À faire**, **En cours**, **Fait**
- Limitation des tâches en cours pour garantir un focus sur les priorités

Rituels d'équipe

- **Réunions de planification** en début de cycle pour définir les objectifs
- **Points réguliers** (daily ou hebdo) pour suivre l'avancement et réajuster

- **Rétrospectives** en fin de cycle pour analyser et améliorer

Cycles de développement

- Projet structuré en **phases itératives**, sans sprints rigides
- Développement, test et intégration des fonctionnalités en continu, selon leur priorité

---

4. Outils et bonnes pratiques

Outils utilisés

- **Jira** : gestion des tâches
- **GitHub** : versioning et collaboration technique
- **Discord** : communication rapide

Bonnes pratiques

- **Priorisation** basée sur la valeur utilisateur
- **Documentation continue** dans les outils de suivi
- **Intégration régulière** pour éviter les conflits de code
- **Revue de code** systématique entre pairs

---

5. Exemple d'application de la méthode

Lors du développement du **module de connexion de la balance connectée** :

- Le tableau Kanban a permis de découper les étapes :
  `Design interface → Communication Bluetooth → Envoi des données → Tests`
- L'équipe a identifié rapidement un **blocage API** en visualisant l'état de la tâche.
- Un membre a été **réaffecté temporairement** pour résoudre ce point, sans ralentir l'avancement global.

5. Méthodologie de développement

Introduction

Dans le cadre du développement de l'application mobile **Mindiff**, une solution de suivi fitness connectée à une balance intelligente, nous avons structuré notre travail autour d'une **méthodologie hybride**, combinant des éléments d'**Agile** et de **Kanban**.

Ce choix vise à travailler de manière **efficace, adaptable et collaborative**, tout en gardant une vision claire de l'avancement.
Ce document expose les fondements de cette approche, les raisons de son adoption, et sa mise en œuvre dans le cadre du projet.

---

1. Principes de la méthode Agile

La méthode **Agile** repose sur **quatre valeurs fondamentales** :

- Les individus et leurs interactions plus que les processus et les outils
- Un logiciel opérationnel plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

Ces valeurs sont complétées par **douze principes**, dont les plus pertinents pour notre projet :

- Livraison fréquente de fonctionnalités opérationnelles
- Implication continue des utilisateurs ou parties prenantes
- Adaptabilité face aux changements
- Coopération quotidienne entre les membres de l'équipe
- Amélioration continue des processus

Cette philosophie permet une organisation **souple, réactive**, centrée sur la valeur pour l'utilisateur final, tout en favorisant **la communication et la transparence**.

---

2. Présentation de Kanban

Kanban est un système visuel de gestion de flux de travail, utilisé initialement dans l'industrie et aujourd'hui adapté au développement logiciel.

Principes de base :

- **Visualiser le flux de travail** via un tableau à colonnes (ex. : À faire / En cours / Fait)
- **Limiter le travail en cours (WIP)** pour éviter la surcharge et garantir la fluidité
- **Mesurer et améliorer en continu** : identifier les blocages, ajuster les processus

Cette approche permet un **suivi en temps réel** des tâches, une **détection rapide des blocages**, et une **optimisation de la charge** selon les ressources disponibles.

---

3. Mise en œuvre de la méthode hybride Agile / Kanban

Organisation du travail

- Utilisation d'un **tableau Kanban** partagé (Trello, Jira, etc.)
- Découpage des tâches en unités fonctionnelles : développement, test, design, intégration
- Trois colonnes principales : **À faire**, **En cours**, **Fait**
- Limitation des tâches en cours pour garantir un focus sur les priorités

Rituels d'équipe

- **Réunions de planification** en début de cycle pour définir les objectifs
- **Points réguliers** (daily ou hebdo) pour suivre l'avancement et réajuster
- **Rétrospectives** en fin de cycle pour analyser et améliorer

Cycles de développement

- Projet structuré en **phases itératives**, sans sprints rigides
- Développement, test et intégration des fonctionnalités en continu, selon leur priorité

---

4. Outils et bonnes pratiques

Outils utilisés

- **Jira** : gestion des tâches
- **GitHub** : versioning et collaboration technique
- **Discord** : communication rapide

Bonnes pratiques

- **Priorisation** basée sur la valeur utilisateur
- **Documentation continue** dans les outils de suivi
- **Intégration régulière** pour éviter les conflits de code
- **Revue de code** systématique entre pairs

---

5. Exemple d'application de la méthode

Lors du développement du **module de connexion de la balance connectée** :

- Le tableau Kanban a permis de découper les étapes :
  `Design interface → Communication Bluetooth → Envoi des données → Tests`
- L'équipe a identifié rapidement un **blocage API** en visualisant l'état de la tâche.

- Un membre a été **réaffecté temporairement** pour résoudre ce point, sans ralentir l’avancement global.

6. Diagramme de processus de développement

Diagramme de processus de développement

7. Softawre Funconnial Requirement

1. Introduction

1.1 Objectif

L'objectif de cette application mobile est de proposer un accompagnement sportif personnalisé en utilisant une intelligence artificielle capable d'analyser les mouvements de l'utilisateur via la caméra. L'application proposera des exercices adaptés selon le pro l de l'utilisateur et suivra ses progrès pour améliorer sa performance et réduire les risques de blessures.

1.2 Technologies Utilisées

- Framework Mobile : Flutter
- Backend & Authenti cation : Firebase
- Base de données : Firestore
- Analyse de mouvements : Modèle IA intégré
- Cloud Storage : Firebase Storage

2. Fonctionnalités

2.1 Authentification et Gestion des Utilisateurs

• Inscription et connexion via e-mail/mot de passe
• Création et gestion du pro l utilisateur avec âge, poids, taille, niveau sportif, et objectifs.
Stockage des préférences et historique des entraînements dans Firestore.

2.2 Suivi et Adaptation des Exercices

• Proposition d'exercices adaptés en fonction du pro l de l'utilisateur.
• Mise à jour automatique du programme en fonction des performances et progrès.
• Intégration d'un calendrier d'entraînement avec rappels.

2.3 Analyse des Mouvements via IA

• Accès à la caméra pour analyser les mouvements de l'utilisateur en temps réel.
• Détection des erreurs de posture et correction en direct.• Stockage des performances et recommandations d'amélioration.

2.4 Interface d'Entraînement

• Affichage des exercices avec animations ou vidéos explicatives.
• Minuteur intégré et instructions vocales pour guider l'utilisateur.
• Feedback instantané basé sur l'analyse IA.
2.5 Suivi des Statistiques• Graphiques et données sur la progression des performances.
• Suivi des séances réalisées et comparaison avec les objectifs.
• Système de récompenses et dé s pour encourager l'utilisateur.
2.6 Notifications et Rappels
• Rappels personnalisés pour les séances d'entraînement.
• Notifications pour les nouveaux dé s et progrès réalisés.

2.7 Menus Personnalisés

• Suggestions de menus alimentaires adaptés aux objectifs de l'utilisateur (prise de masse, perte de poids, maintien).
• Intégration de recommandations nutritionnelles basées sur l'activité physique enregistrée.
• Suivi des apports caloriques et des macronutriments en lien avec les séances d'entraînement.

2.8 Intégration d'une Balance Connectée

• Connexion avec une balance connectée via Bluetooth ou Wi-Fi.
• Suivi du poids, de l'IMC et de la composition corporelle (masse grasse, masse

musculaire, hydratation).
• Synchronisation des données avec le pro l de l'utilisateur.
• Ajustement des recommandations nutritionnelles et des exercices en fonction des mesures enregistrées.

3. Contraintes et Exigences

• L'application doit fonctionner sur iOS et Android.
• La gestion des données doit respecter le RGPD et garantir la sécurité des informations.
• L'analyse en temps réel doit être optimisée pour éviter une consommation excessive de batterie.
• L'application doit être accessible avec une connexion internet et permettre un mode hors-ligne limité.

4. Cas d'Utilisation

Un utilisateur crée un compte et remplit son pro l.
L'application lui propose un programme d'exercices personnalisé.
L'utilisateur effectue un exercice sous le suivi de l'IA via la caméra.
L'application analyse ses mouvements et donne un feedback en temps réel.
L'utilisateur termine la séance et consulte ses statistiques.
Il reçoit des recommandations pour la prochaine session.

5. Évolutions Possibles• Intégration d'une communauté pour partager ses performances.

- Compatibilité avec des objets connectés (montres, capteurs).
- Ajout d'un coach virtuel interactif.
- Gamification avec des classements et défis entre amis.

8. Document d'intégration de l'équipe IA

Objectif du document

Ce document vise à faciliter la **prise en main du module d'Intelligence Artificielle (IA)** de l'application mobile **Mindiff**, dédiée au suivi fitness par analyse de mouvements en temps réel via la caméra.
Il s'adresse aux **nouveaux développeurs IA** et fournit les éléments nécessaires pour :

- Comprendre l'architecture existante,

- Maintenir le code,
- Faire évoluer l’intégration de **MediaPipe** pour l’analyse des exercices.

Présentation générale

Le module IA analyse les **mouvements sportifs en direct** (squat, développé couché, etc.) via la caméra de l'utilisateur.
Il repose sur **MediaPipe Pose Landmarker** pour :

- **Détecter les points clés du corps**,
- **Évaluer la qualité des mouvements**,
- **Fournir un feedback correctif en temps réel**.

Architecture globale :

1. Flux vidéo capté depuis la caméra.
2. Analyse en temps réel via MediaPipe.
3. Extraction des points de pose.
4. Traitement des données pour chaque exercice.
5. Génération et affichage du feedback à l'utilisateur.

Pré-requis techniques

Outils de test :

- Émulateur ou **appareil mobile avec caméra**.
- **Vidéos de référence** : bonnes et mauvaises exécutions.
- Outils de **visualisation de données de pose** (disponibles ou à développer).

Procédure d’intégration

Étapes pour intégrer MediaPipe dans l’app :

1. Initialiser **MediaPipe Pose Landmarker**.
2. Brancher le **flux caméra** en entrée du pipeline.
3. Récupérer les **points clés de pose** pour chaque frame.
4. **Parser & traiter** ces données (angles, distances, vitesses).
5. Appliquer la **logique métier** spécifique à l’exercice.
6. Générer un **feedback clair et instantané**.
7. **Afficher** ce feedback dans l’interface en temps réel.

Données échangées

Types de données traitées par l'IA :

- **Coordonnées des points clés du corps** (x, y, z + score de confiance)
- **Timestamp implicite** (frame par frame)

Traitement :

- **Vérification de la validité** des points (score de confiance).
- **Calcul de métriques** (angles, distances).
- **Détection d'erreurs** en comparant aux seuils préétablis.

Gestion des erreurs

Cas à gérer :

- **Perte de suivi** (occlusion, mouvements rapides).
- **Points clés peu fiables** : score de confiance faible.
- **Erreurs de calculs** des métriques : validation requise.
- **Logique d'évaluation incorrecte** : tests unitaires & d'intégration.
- **Problèmes de performance** : optimisation nécessaire pour analyse fluide.

Outils de débogage recommandés :

- **Logs intégrés** (console, fichier, UI debug).
- **Superposition des points clés** sur la vidéo.
- **Outils de profilage** pour identifier les ralentissements.

Bonnes pratiques

- Lire la **documentation MediaPipe** (limitations & best practices).
- **Optimiser les traitements** pour la performance temps réel.
- **Modulariser par exercice** la logique d'analyse.
- Implémenter des **tests unitaires** pour la logique de métriques.
- Utiliser un **jeu de données varié** pour la robustesse IA.
- **Logger intelligemment** les infos critiques (pose brute, erreurs, métriques calculées).

9. Intégration équipe IA

Objectif du document

Ce document vise à faciliter la **prise en main du module d'Intelligence Artificielle (IA)** de l'application mobile **Mindiff**, dédiée au suivi fitness par analyse de mouvements en temps réel via la caméra.
Il s'adresse aux **nouveaux développeurs IA** et fournit les éléments nécessaires pour :

- Comprendre l'architecture existante,
- Maintenir le code,
- Faire évoluer l'intégration de **MediaPipe** pour l'analyse des exercices.

Présentation générale

Le module IA analyse les **mouvements sportifs en direct** (squat, développé couché, etc.) via la caméra de l'utilisateur.
Il repose sur **MediaPipe Pose Landmarker** pour :

- **Détecter les points clés du corps**,
- **Évaluer la qualité des mouvements**,
- **Fournir un feedback correctif en temps réel**.

Architecture globale :

1. Flux vidéo capté depuis la caméra.
2. Analyse en temps réel via MediaPipe.
3. Extraction des points de pose.
4. Traitement des données pour chaque exercice.
5. Génération et affichage du feedback à l'utilisateur.

Pré-requis techniques

Outils de test :

- Émulateur ou **appareil mobile avec caméra**.
- **Vidéos de référence** : bonnes et mauvaises exécutions.
- Outils de **visualisation de données de pose** (disponibles ou à développer).

Procédure d'intégration

Étapes pour intégrer MediaPipe dans l'app :

1. Initialiser **MediaPipe Pose Landmarker**.

2. Brancher le **flux caméra** en entrée du pipeline.
3. Récupérer les **points clés de pose** pour chaque frame.
4. **Parser & traiter** ces données (angles, distances, vitesses).
5. Appliquer la **logique métier** spécifique à l'exercice.
6. Générer un **feedback clair et instantané**.
7. **Afficher** ce feedback dans l'interface en temps réel.

Données échangées

Types de données traitées par l'IA :

- **Coordonnées des points clés du corps** (x, y, z + score de confiance)
- **Timestamp implicite** (frame par frame)

Traitement :

- **Vérification de la validité** des points (score de confiance).
- **Calcul de métriques** (angles, distances).
- **Détection d'erreurs** en comparant aux seuils préétablis.

Gestion des erreurs

Cas à gérer :

- **Perte de suivi** (occlusion, mouvements rapides).
- **Points clés peu fiables** : score de confiance faible.
- **Erreurs de calculs** des métriques : validation requise.
- **Logique d'évaluation incorrecte** : tests unitaires & d'intégration.
- **Problèmes de performance** : optimisation nécessaire pour analyse fluide.

Outils de débogage recommandés :

- **Logs intégrés** (console, fichier, UI debug).
- **Superposition des points clés** sur la vidéo.
- **Outils de profilage** pour identifier les ralentissements.

Bonnes pratiques

- Lire la **documentation MediaPipe** (limitations & best practices).
- **Optimiser les traitements** pour la performance temps réel.
- **Modulariser par exercice** la logique d'analyse.

- Implémenter des **tests unitaires** pour la logique de métriques.
- Utiliser un **jeu de données varié** pour la robustesse IA.
- **Logger intelligemment** les infos critiques (pose brute, erreurs, métriques calculées).

10. Intégration équipe IOT

Projet Mindiff - IoT – Intégration de la balance connectée

1. Objectif du document
   Ce document a pour but de faciliter la prise en main de la partie IoT du projet Mindiff, une application mobile de suivi fitness connectée à une balance intelligente. Il s'adresse à tout nouveau développeur rejoignant l'équipe, et fournit les éléments nécessaires pour comprendre, maintenir et faire évoluer l'intégration de la balance avec l'application.
2. Présentation générale
   La balance connectée a pour rôle de collecter des données corporelles (poids, masse graisseuse, IMC, etc.) et de les transmettre à l'application Mindiff pour traitement, affichage et suivi.
   Architecture globale :
   La balance communique avec l'application mobile via le bluetooth (BLE) et/ou Wifi.
   L'application se charge de recevoir, parser et stocker les données.
   Ces données sont ensuite affichées à l'utilisateur et peuvent être synchronisées avec une base de données distante.
3. Pré-requis techniques
   Outils de test :
   Émulateur ou appareil mobile
   Balance physique ou simulateur de messages MQTT
4. Procédure d'intégration
   Étapes pour mettre en place l'intégration :
5. Initialiser la connexion Wi-Fi avec le réseau configuré pour la balance.
6. Établir une connexion sans fil (local ou distant).
7. Se connecter à l'application pour publier les mesures.
8. Recevoir et parser les messages (ex : poids en kg, body fat en %, etc.)
9. Stocker temporairement les données, ou les transmettre à l'API centrale.
10. Données échangées
    Les données envoyées par la balance doivent suivre un format structuré.
    Le parsing doit inclure :

Vérification du type et de la validité des données

Gestion du timestampProtection contre les duplications ou messages partiels

11. Gestion des erreurs

    Cas à prévoir :

    Échec de connexion sans fil

    Reconnexion automatique

    Affichage d'une alerte dans l'interface

    Message mal formé

    Rejet du message + log d'erreur

    Perte de connexion Wi-Fi

    Bufferisation éventuelle sur la balance (si supporté)

    Reprise automatique de la connexion

    Outils de débogage recommandés :

    Logs intégrés à l'application mobile (console, fichier, UI debug)

12. Bonnes pratiques

    Ne jamais laisser de credentials en clair dans le code

    Vérifier la validité de chaque message reçu

    Implémenter un mécanisme de reconnexion robuste

    Logger les erreurs de manière exploitable

    Tester sur réseau local et via une connexion réelle